

Structured Learning and Decision-Focused Optimization: From Prediction to Decisions in Finance

Wouter van Heeswijk

May 14, 2026

MSCA DIGITAL Deep Learning in Finance

Figueira da Foz, University of Coimbra

Machine learning is increasingly used to make financial decisions.

Examples include:

- Portfolio optimization
- Trading and execution
- Credit allocation
- Risk management
- Market making

But most ML systems are trained to predict — not to decide.

The Key Idea of This Lecture

Can we train machine learning models directly for decision quality?

Instead of: Data $\rightarrow$ Prediction $\rightarrow$ Optimization

we will study: Data $\rightarrow$ **Decision-Aware Learning** $\rightarrow$ Better Decisions

Core themes:

- Structured learning
- Differentiable optimization
- Decision-focused learning
- Geometry-aware losses

Why Learn Decisions Instead of Re-Solving Optimization?

In many financial systems, repeatedly solving large optimization problems can be too slow or expensive.

Examples:

- high-frequency trading
- real-time portfolio adjustment
- dynamic execution
- large-scale routing or allocation

Structured learning aims to learn fast decision rules that approximate expensive optimization procedures, mimicking the solutions of high-quality solvers.

This also enables:

- anticipatory decisions under uncertainty
- end-to-end adaptation to downstream objectives
- scalable real-time inference

Why Is This Challenging?

Optimization problems are often:

- discrete
- constrained
- combinatorial
- non-differentiable

Modern ML relies on:

- gradients
- smoothness
- backpropagation

Central Tension

How do we combine optimization and deep learning?

What You Should Take Away From This Lecture

By the end of this lecture, you should understand:

- Why prediction accuracy and decision quality differ
- What structured prediction means
- Why optimization layers are difficult to train
- How regularization creates differentiability
- The intuition behind:
 - Fenchel conjugates
 - Fenchel–Young losses
 - Bregman divergences
- How these ideas apply to finance

Roadmap

- 1 From Prediction to Decision
- 2 Structured Prediction and Optimization Layers
- 3 Smoothing Structured Optimization
- 4 Fenchel–Young Losses and Bregman Geometry
- 5 Learning Algorithms and Modern Methods
- 6 Applications in Finance

From Prediction to Decision

Most quantitative systems follow the same architecture:

Data $\rightarrow$ Prediction Model $\rightarrow$ Optimization $\rightarrow$ Decision

Examples:

- Predict returns $\rightarrow$ optimize portfolios
- Predict volatility $\rightarrow$ allocate risk
- Predict order flow $\rightarrow$ execute trades

Machine learning and optimization are usually treated as separate modules.

The Classical Paradigm

Step 1: Prediction

Train a statistical model:

$$\hat{\xi} = \phi_w(x)$$

where:

- x = market features
- ξ = unknown future quantities

Step 2: Optimization

Use predictions inside a decision problem:

$$y^*(x) = \arg \min_{y \in Y} c(x, y, \hat{\xi})$$

Implicit assumption:

- Better predictions $\Rightarrow$ better decisions

But Prediction Accuracy Is Not the Real Objective

Prediction models are trained using statistical losses

Examples:

MSE, cross-entropy, likelihood

But portfolio managers care about:

- returns
- risk
- drawdowns
- transaction costs
- tail exposure

Core Observation

Prediction quality and decision quality are not the same thing.

A Simple Finance Example

Suppose two models have identical prediction error.

Model A

- Slightly underestimates crash risk

Model B

- Slightly overestimates volatility under normal market conditions

Under MSE:

$$\text{Model A} \approx \text{Model B}$$

For portfolio optimization:

$$\text{Model A} \ll \text{Model B}$$

Optimization amplifies certain prediction errors.

Why Optimization Amplifies Errors

Optimization is highly non-linear in predictions

Small perturbations in $\hat{\xi}$ can produce large changes in $y^*(x)$

Examples:

- Portfolio concentration
- Leverage explosions
- Instability under covariance estimation
- Excessive turnover

The optimizer exploits whatever signal it is given.

Good predictions do not necessarily induce good decisions.

Good decisions may not require accurate predictions everywhere.

Key idea:

- Some errors matter much more than others
- Learning should focus on decision-relevant information

Running Example: Portfolio Optimization

State (context)

x = market information

Transition (uncertainty)

ξ = future returns

Action (allocation decision)

$y \in Y$ = portfolio weights

subject to:

$$\sum_i y_i = 1, \quad y_i \geq 0$$

Goal

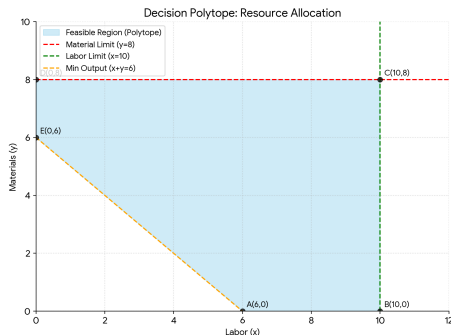
$$\min_{y \in Y} \mathbb{E}[c(x, y, \xi)]$$

The Decision Space Is Structured

Portfolio decisions are not arbitrary vectors.

They must satisfy constraints:

- Budget constraints
- Risk limits
- Sector constraints
- Leverage bounds
- Cardinality constraints



Important

The output is a structured object such as a polytope.

The learning problem is therefore not standard classification or regression.

From Scalar Outputs to Structured Outputs

Classical ML:

$$f(x) \in \mathbb{R}$$

Structured learning:

$$y(x) \in \mathcal{Y}$$

where:

$\mathcal{Y}$ = set of feasible decisions

Examples:

- Portfolios
- Matchings
- Paths
- Schedules
- Trading trajectories

A Different View of Machine Learning

Instead of predicting decisions directly:

$$\theta = \phi_w(x)$$

The neural network produces scores or preferences.

A solver then computes the final decision:

$$y(\theta) = \arg \max_{y \in \mathcal{Y}} \theta^\top y$$

Optimization enforces feasibility and structure.

Input $\rightarrow$ Neural Network $\rightarrow$ Optimization Problem $\rightarrow$ Decision

This architecture is called:

- Decision-focused learning
- Predict-then-optimize
- Combinatorial optimization augmented ML

The optimization problem becomes part of the model itself.

Example of COAML pipeline

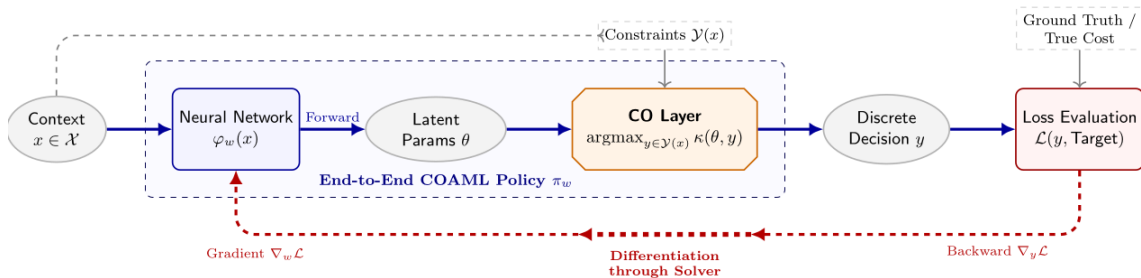


Figure: Conceptual illustration of the COAML paradigm. A neural network (NN) parameterizes a surrogate combinatorial optimization (CO) problem. We propagate gradients backwards through the CO layer, enabling end-to-end training of the policy π_w to minimize a down-stream decision loss.

Source: *Combinatorial Optimization Augmented Machine Learning*, Schiffer et al. (2026).

A Key Conceptual Shift

Classical ML asks:

“Can we predict the future accurately?”

Decision-focused learning asks:

“Can we make good decisions under uncertainty?”

These are fundamentally different objectives.

What Makes This Difficult?

We would like to train $\pi_w(x)$ using gradient-based learning.

But decisions are generated through an arg max over a discrete decision space.

Problem:

- Discrete optimization is not differentiable
- Small changes in scores may not change the solution
- Gradients become zero almost everywhere

Structured Prediction and Optimization Layers

Why Structured Outputs Appear Naturally

As mentioned, decisions are not unconstrained vectors. They must satisfy:

- feasibility constraints
- logical consistency
- combinatorial structure

Examples in finance:

- Portfolio weights sum to one
- Execution trajectories satisfy inventory constraints
- Trading schedules obey market impact limits
- Asset allocations satisfy regulatory constraints

Key Point

The structure of the decision matters.

Examples of Structured Outputs

Finance

- Portfolios
- Trading trajectories
- Asset allocations
- Execution schedules

Operations Research

- Vehicle routes
- Matchings
- Schedules
- Supply chains

Machine Learning

- Parsing
- Sequence labeling
- Image segmentation
- Graph prediction

Common Feature

- Outputs satisfy global constraints

Structured Prediction Formulation

The prediction rule becomes:

$$y(\theta) = \arg \max_{y \in \mathcal{Y}} \theta^\top y$$

Interpretation

- Neural network proposes scores
- Optimization layer enforces feasibility
- Final output is a structured decision

The solver becomes part of the model itself.

A Portfolio Interpretation

Suppose:

θ_i = attractiveness score of asset i

The optimizer solves:

$$\max_{y \in Y} \theta^\top y$$

subject to:

- budget constraints
- leverage limits
- sector exposure bounds
- turnover constraints

The network does not output the portfolio directly.

Optimization as an Inductive Bias

The optimization layer injects domain knowledge.

Examples:

- Feasibility
- Risk control
- Sparsity
- Diversification
- Regulatory requirements

Important

Optimization acts as a structured inductive bias.

This is one reason structured learning is powerful.

Combinatorial Optimization Augmented ML (COAML)

General architecture:

$$x \rightarrow \phi_w(x) \rightarrow \text{Optimization Problem} \rightarrow y$$

The optimization problem may involve:

- linear programming
- integer programming
- shortest paths
- matching
- routing

Optimization is embedded inside the learning pipeline.

Why End-to-End Learning Is Difficult

The prediction rule is $y(\theta) = \arg \max_{y \in \mathcal{Y}} \theta^\top y$

The optimizer selects the best discrete solution.

Problem:

- small changes in θ often do not change the optimizer at all
- then suddenly the solution jumps to another decision

As a result, the prediction map is:

- non-smooth
- piecewise constant
- non-differentiable

Central Problem

How can we backpropagate through optimization problems?

Geometric Interpretation

The optimizer selects vertices of a feasible polytope.

As long as θ stays inside the same normal cone, the optimal vertex does not change.

Therefore:

$$\frac{\partial y(\theta)}{\partial \theta} = 0 \quad \text{almost everywhere}$$

Then, at cone boundaries, the optimizer abruptly switches to another vertex.

Result:

- unstable gradients
- poor credit assignment
- difficult end-to-end learning

Where We Are Going Next

We now face a fundamental conflict:

- Optimization layers are discrete
- Deep learning requires smoothness

The key idea of modern structured learning is:

Smooth the optimization problem itself.

This leads to:

- regularized prediction
- Fenchel conjugates
- Fenchel–Young losses
- Bregman geometry

Smoothing Structured Optimization

Geometry of Structured Prediction

Let $\mathcal{C} = \text{conv}(\mathcal{Y})$ be the convex hull of feasible solutions.

Important observation:

$$\max_{y \in \mathcal{Y}} \theta^\top y = \max_{y \in \mathcal{C}} \theta^\top y$$

Interpretation:

- Linear optimization always selects extreme points
- Structured prediction can be viewed geometrically
- The feasible set becomes a polytope

Why the Prediction Map Is Discontinuous

The optimizer selects vertices of the polytope $\mathcal{C}$.

As long as θ remains inside the same normal cone, the solution does not change.

Then suddenly $y(\theta)$ jumps to another vertex.

Result:

- piecewise constant predictions
- discontinuities at cone boundaries

Cone on Solution Polytope

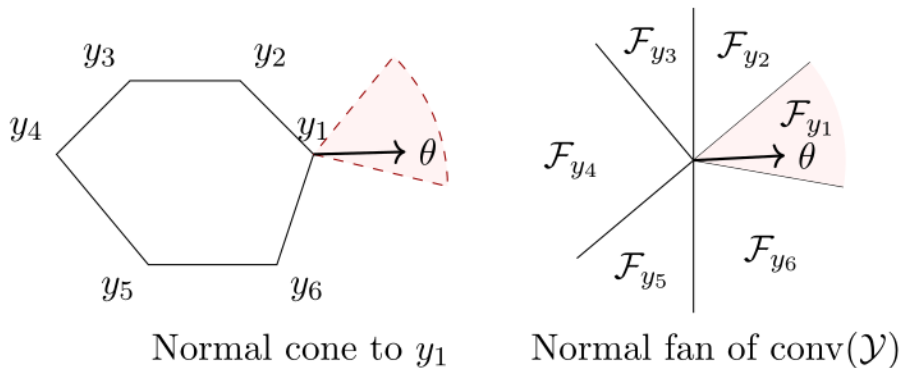


Figure: The oracle is piecewise constant on each cone of the fan. Without regularization, the solution jumps from one vertex to another when crossing a certain boundary for θ

Source: *Combinatorial Optimization Augmented Machine Learning*, Schiffer et al. (2026).

Why the Hard Optimizer Is Problematic

Recall:

$$y(\theta) = \arg \max_{y \in \mathcal{C}} \theta^\top y$$

Linear optimization prefers extreme points of the feasible set.

Consequence:

- solutions jump abruptly between vertices
- tiny changes in θ can change the optimizer completely
- gradients become unstable or vanish

Adding a Regularizer

A regularizer assigns a penalty to certain decisions.

Instead of maximizing only performance $\theta^\top y$ we also penalize undesirable behavior.

Examples in finance:

- concentrated portfolios
- excessive leverage
- unstable allocations
- high turnover

Interpretation:

good decision = high reward – complexity/stability penalty

The Key Idea: Regularized Optimization

Instead of solving:

$$\max_{y \in \mathcal{C}} \theta^\top y$$

we solve:

$$\max_{y \in \mathcal{C}} \left(\theta^\top y - \Omega(y) \right)$$

where $\Omega(y)$ is a strongly convex regularizer.

Effect of the regularizer:

- discourages brittle extreme solutions
- rewards smoother or more stable decisions
- makes the optimizer less sensitive to perturbations

What Does Strong Convexity Do?

Without regularization:

- linear optimization selects vertices
- solutions change discontinuously

Strong convexity adds curvature to the objective and progressively penalizes extreme solutions.

This means the optimizer now prefers:

- stable optima
- smoother changes as θ varies

Key Effect

The prediction map becomes continuous and differentiable.

Regularized Prediction Map

We regularize the prediction map:

$$\hat{y}(\theta) = \arg \max_{y \in \mathcal{C}} \left(\theta^\top y - \Omega(y) \right)$$

This replaces the hard prediction map:

$$y(\theta)$$

by a smooth alternative:

$$\hat{y}(\theta)$$

Goal:

- recover meaningful gradients
- enable end-to-end learning

From Optimization to Differentiability

Recall the regularized prediction rule:

$$\hat{y}(\theta) = \arg \max_{y \in \mathcal{C}} \left(\theta^\top y - \Omega(y) \right)$$

We now ask:

How does the optimal solution change when the scores θ change?

To study this, we define the *optimal value* of the optimization problem:

$$\Omega^*(\theta) = \max_{y \in \mathcal{C}} \left(\theta^\top y - \Omega(y) \right)$$

Interpretation:

- for each score vector θ , $\Omega^*(\theta)$ gives the best achievable objective value
- this function is called the **Fenchel conjugate** or convex conjugate

Visualisation of Fenchel Conjugate

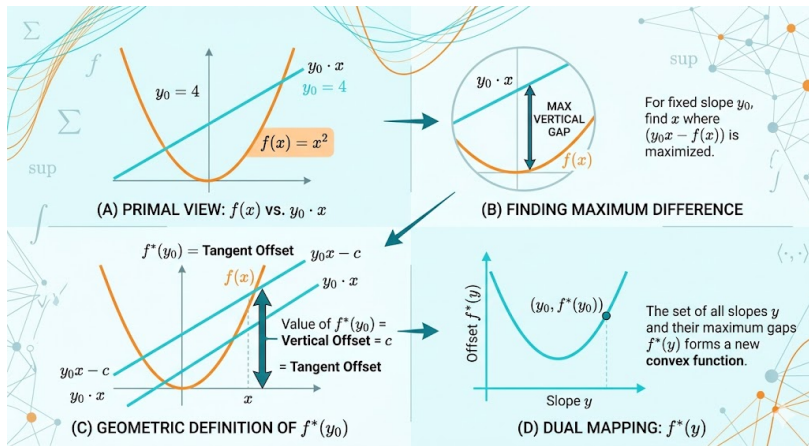


Figure: The Fenchel conjugate summarizes the value of the regularized optimization problem as a function of the scores θ . Its gradient recovers the smooth prediction map.

Why the Fenchel Conjugate Matters

A remarkable identity holds:

$$\hat{y}(\theta) = \nabla \Omega^*(\theta)$$

Interpretation:

- decision predictions become gradients of a convex function
- the prediction map is now smooth

This is the bridge between:

- optimization
- geometry
- differentiable learning

What Geometry Does the Regularizer Induce?

Different regularizers produce different geometries.

Examples:

- Quadratic regularization

$$\Omega(y) = \frac{1}{2} \|y\|^2$$

induces Euclidean geometry

- Entropic regularization

$$\Omega(y) = \sum_i y_i \log y_i$$

induces probabilistic geometry

The choice of regularizer defines what “closeness” means.

Where We Are Now

We started with:

$$\arg \max_{y \in \mathcal{Y}} \theta^\top y$$

Problem:

- discontinuous predictions
- zero gradients

Solution:

- smooth the optimization problem
- obtain differentiable prediction maps

Next:

- how do we define principled loss functions?

Fenchel–Young Losses and Bregman Geometry

From Smooth Prediction to Learning

We introduced the smooth prediction map:

$$\hat{y}(\theta) = \nabla \Omega^*(\theta)$$

Regularization solved the differentiability problem.

But learning still requires a loss function.

How should we measure prediction error?

The key idea: the loss should be compatible with the geometry induced by the regularizer.

Fenchel–Young Losses

Given the regularized prediction map

$$\hat{y}(\theta) = \nabla \Omega^*(\theta)$$

we define the Fenchel–Young loss:

$$\mathcal{L}_{FY}(\theta, \bar{y}) = \Omega^*(\theta) + \Omega(\bar{y}) - \theta^\top \bar{y}$$

where:

- $\hat{y}(\theta)$ is the prediction
- $\bar{y}$ is the target decision

Interpretation:

The loss measures how far the target decision is from the prediction induced by the current scores θ .

Why Fenchel–Young Losses Matter

A remarkable identity holds:

$$\nabla_{\theta} \mathcal{L}_{FY}(\theta, \bar{y}) = \hat{y}(\theta) - \bar{y}$$

The gradient has the familiar form:

prediction – target

But now the prediction $\hat{y}(\theta)$ comes from a structured optimization problem.

Key Insight

Optimization layers become compatible with backpropagation.

The Geometry Behind Fenchel–Young Losses

Fenchel–Young losses can be interpreted as Bregman divergences:

$$\mathcal{L}_{FY}(\theta, \bar{y}) = D_{\Omega}(\bar{y} \mid \hat{y}(\theta))$$

where

$$D_{\Omega}(y \mid \bar{y}) = \Omega(y) - \Omega(\bar{y}) - \nabla \Omega(\bar{y})^{\top} (y - \bar{y})$$

Interpretation: Bregman divergence measures the gap between a convex function and its tangent approximation.

The regularizer therefore defines:

- the prediction map
- the geometry of prediction error

Visualization of Bregman Divergence

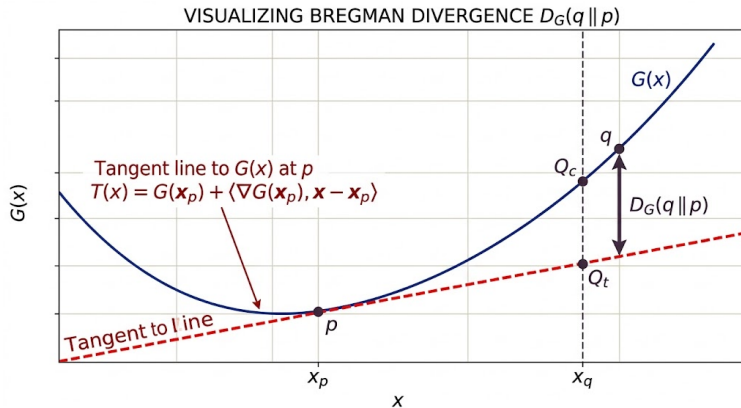


Figure: A Bregman divergence measures the gap between a convex function and its tangent approximation at a reference point. Unlike Euclidean distance, the discrepancy is induced by the geometry of the regularizer itself.

Big Picture: What Each Concept Does

- **Regularizer** $\Omega(y)$ Adds curvature to the optimization problem:

$$\max_y \theta^\top y \rightarrow \max_y \theta^\top y - \Omega(y)$$

Effect: smoother and more stable decisions.

- **Fenchel conjugate** $\Omega^*(\theta)$ The value function of the regularized optimization problem:

$$\Omega^*(\theta) = \max_y (\theta^\top y - \Omega(y))$$

Its gradient gives the prediction $\nabla \Omega^*(\theta) = \hat{y}(\theta)$

- **Fenchel–Young loss** A training objective compatible with backpropagation:

$$\nabla_{\theta} \mathcal{L}_{FY} = \hat{y}(\theta) - \bar{y}$$

- **Bregman divergence** The geometric interpretation of prediction error:

true value – tangent approximation

The regularizer defines the geometry of the loss.

Quadratic Regularization

Suppose we use quadratic regularizer $\Omega(y) = \frac{1}{2}\|y\|^2$

Then the associated Bregman divergence becomes $D_{\Omega}(y \mid \bar{y}) = \frac{1}{2}\|y - \bar{y}\|^2$

Interpretation:

- the tangent approximation error becomes ordinary squared distance
- curvature is identical in every direction
- nearby decisions are measured using standard Euclidean geometry

Key Insight

Quadratic regularization recovers classical least-squares learning as a special case of Fenchel–Young geometry.

Entropic Regularization

Suppose we use entropic regularizer

$$\Omega(y) = \sum_i y_i \log y_i$$

Entropy discourages concentrated vertex solutions and favors diffuse probability distributions.

The induced prediction map becomes:

$$\hat{y}(\theta) = \text{softmax}(\theta)$$

The associated Bregman divergence is:

$$D_{\Omega}(y \mid \bar{y}) = \text{KL}(y \parallel \bar{y})$$

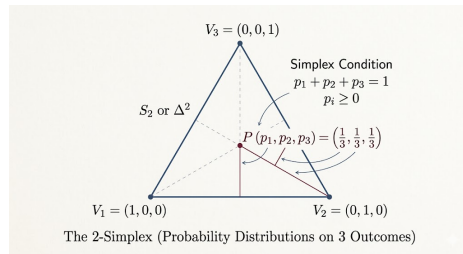


Figure: Entropy pushes solutions away from simplex vertices, smoothing hard argmax decisions into soft probabilistic assignments.

A Unifying Perspective

Many familiar ML losses arise as special cases of Fenchel–Young losses.

Examples:

- Squared loss
- Logistic loss
- Cross-entropy
- Sparsemax losses

Key Insight

Modern machine learning losses can be viewed through a lens of convex duality and structured optimization.

Why This Matters for Finance

Financial decisions are highly geometry-dependent.

Different regularizers induce different behavior:

- Euclidean geometry $\rightarrow$ smooth allocations
- Entropic geometry $\rightarrow$ diversified probabilistic allocations
- Sparse regularization $\rightarrow$ concentrated portfolios

The regularizer encodes financial preferences and risk structure.

Where We Go Next

We now have:

- differentiable prediction maps
- principled geometry-aware losses

Next question:

How do we actually train these systems in practice?

Learning Algorithms and Modern Methods

From Geometry to Computation

So far, we established that regularization can transform structured optimization problems into differentiable prediction maps.

In particular:

$$\hat{y}(\theta) = \nabla \Omega^*(\theta)$$

allows gradients to propagate through optimization layers.

The remaining question is computational:

How do we compute these gradients efficiently in practice?

Why Do We Need Approximation Methods?

The framework is mathematically elegant, but computationally expensive.

During training, gradients must propagate through optimization problems repeatedly.

This may require:

- solving optimization problems thousands of times
- differentiating through iterative solvers
- repeated backward passes through large optimization layers

For large-scale structured or combinatorial problems, exact differentiation quickly becomes impractical.

Key Consequence

Modern methods often trade exact gradients for efficient approximations.

So far, we focused on one strategy:

Smooth the optimization problem itself.

This introduced:

- regularization
- Fenchel conjugates
- smooth prediction maps
- Fenchel–Young losses

Modern decision-focused learning therefore contains multiple algorithmic strategies for recovering gradients through optimization problems.

Three Computational Strategies

Modern decision-focused learning uses different approaches depending on the structure of the optimization problem.

- 1 **Convex optimization**

Differentiate the optimality conditions directly.

- 2 **Discrete linear optimization**

Replace the non-differentiable objective with a surrogate loss.

- 3 **Black-box combinatorial optimization**

Smooth the solver using perturbations or randomization.

Common goal: recover useful gradients for end-to-end learning.

1. Convex Optimization: Implicit Differentiation

Suppose the optimization problem is sufficiently smooth and satisfies suitable regularity conditions. At the optimum, the solution satisfies a system of equations (for example, the KKT conditions):

$$F(y^*, \theta) = 0$$

The key idea:

Differentiate the optimality conditions themselves.

Using the implicit function theorem:

$$\frac{dy^*}{d\theta} = -\nabla_y F(y^*, \theta)^{-1} \nabla_\theta F(y^*, \theta)$$

This produces exact gradients through the optimizer.

2. Discrete Optimization: Surrogate Loss

Consider the discrete optimization problem $y^*(\xi) = \arg \min_{y \in Y} \xi^\top y$, where ξ are the true costs.

A predictive model produces estimated costs $\hat{\xi}$, leading to the decision:

$$\hat{y} = y^*(\hat{\xi})$$

The true objective is not accurate prediction, but good downstream decisions. This can be measured using **decision regret**:

$$\xi^\top \hat{y} - \xi^\top y^*(\xi)$$

which quantifies how much worse the predicted decision is compared to the true optimum.

Problem: regret is still non-differentiable because the optimizer is discrete.

Key idea: construct a differentiable surrogate loss that approximates regret while still providing useful gradients for learning.

3. The Combinatorial Case: Smoothing

Some combinatorial solvers are difficult to differentiate directly. Instead, we smooth the optimization problem using random perturbations.

Logic: If a single solver call is a step function, then the **average** of many calls with slight random noise is smooth.

$$\hat{y}(\theta) = \mathbb{E}_Z[\text{Solver}(\theta + \epsilon Z)]$$

This allows us to differentiate through, e.g., max-flow or matching solvers.

The Big Picture

Decision-focused learning combines:

- optimization
- convex geometry
- differentiable learning
- structured decision making

The central idea throughout this lecture was:

Make optimization compatible with gradients.

Regularization, conjugates, Fenchel–Young losses, and modern differentiation methods are all different parts of this same objective.

Applications in Finance

Why Applications Matter

Decision-focused learning is not only a theoretical framework.

It is motivated by real systems where:

- predictions drive decisions
- optimization amplifies errors
- constraints are essential

These problems appear naturally in:

- finance
- logistics
- transportation
- supply chains
- energy systems

Portfolio Optimization

Classical pipeline:

Predict returns $\rightarrow$ Optimize portfolio

Typical optimization problem (Markowitz portfolio optimization is a quadratic programming problem):

$$\max_{y \in Y} \mu^\top y - \lambda y^\top \Sigma y$$

where:

- μ = expected returns
- Σ = covariance matrix
- y = portfolio weights

Why Portfolio Optimization Is Difficult

Portfolio optimization is highly sensitive to estimation error.

Small errors in μ can produce:

- concentrated portfolios
- unstable allocations
- excessive turnover
- leverage explosions

Key Observation

The optimizer aggressively exploits noisy predictions.

Decision-Focused Portfolio Learning

Instead of learning:

$$\hat{\mu} \approx \mu$$

we learn predictions that induce good portfolios.

Training objective:

$$\min_w \mathbb{E} [c(x, \pi_w(x), \xi)]$$

Examples of decision-aware objectives:

- Sharpe ratio
- drawdown control
- turnover penalties
- CVaR objectives

Regularization in Portfolio Problems

Regularization has natural financial interpretations.

Examples:

- ℓ_2 regularization $\rightarrow$ diversification
- entropy regularization $\rightarrow$ smoother allocations
- sparse regularization $\rightarrow$ concentrated portfolios
- turnover penalties $\rightarrow$ trading stability

The geometry of the optimizer encodes investment preferences.

Trade Execution

Large orders must be executed gradually.

Goals include:

- minimizing market impact
- minimizing slippage
- controlling inventory risk

Typical structured decision:

y = execution trajectory

subject to:

- inventory constraints
- liquidity constraints
- time consistency

Market Making

Market makers continuously choose:

- bid quotes
- ask quotes
- stock positions

Objectives involve:

- profitability
- inventory risk
- adverse selection

The problem becomes:

- sequential
- stochastic
- constrained

This strongly connects structured optimization and reinforcement learning.

Risk Management

Risk systems also involve structured decisions.

Examples:

- hedging strategies
- capital allocation
- stress testing
- liquidity management

Important characteristics:

- tail risks dominate outcomes
- prediction errors are asymmetric
- robustness is critical

Decision-focused methods naturally emphasize downstream consequences.

Why This Research Area Is Growing

Several trends are converging:

- larger datasets
- better optimization solvers
- differentiable programming
- reinforcement learning
- increased automation of decision systems

Modern ML systems increasingly need to:

make decisions, not just predictions.

Current Research Challenges

Important open problems include:

- scalability for large combinatorial problems
- robustness under distribution shift
- interpretability
- safe exploration in RL
- non-convex optimization layers
- uncertainty quantification

Many of these challenges are especially important in finance.



Schiffer, M., Hoppe, H., Su, Y., Bouvier, L., & Parmentier, A. (2026).
Combinatorial Optimization Augmented Machine Learning.
arXiv preprint [arXiv:2601.10583](https://arxiv.org/abs/2601.10583).

Final Takeaways

- Prediction and decision quality are fundamentally different
- Structured outputs naturally arise in finance and OR
- Optimization layers create differentiability challenges
- Regularization and convex geometry provide smooth learning frameworks
- Fenchel–Young losses unify optimization and learning
- Modern methods enable end-to-end decision-focused systems

Machine learning is increasingly evolving from prediction to decision-making.

Thank You

Questions?

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or Horizon Europe: Marie Skłodowska-Curie Actions. Neither the European Union nor the granting authority can be held responsible for them. This project has received funding from the Horizon Europe research and innovation programme under the Marie Skłodowska-Curie Grant Agreement No. 101119635